

2025-1

Basic Algorithm Study For ■ Beginners

기초 알고리즘 스터디 2주차

01

리스트

- 리스트 선언하기
- 인덱싱
- 메소드

02

튜플

- 튜플 선언하기
- 튜플 사용하기

03

딕셔너리, 세트

- 선언하기
- 메소드

04

함수

- 함수가 필요한 이유
- 재귀
- 익명 함수

리스트

—



Q) 기초스터디원 30명의 백준 핸들을 관리하고 싶다.
어떻게 해야 할까?

1.

```
student_0 = 'asdf'
```

```
student_1 = 'qwerty'
```

```
student_2 = 'hiarc'
```

...

```
student_29 = 'OTL'
```

2.

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```


01

리스트 선언하기

Q) 기초스터디원 30명의 백준 핸들을 관리하고 싶다.
어떻게 해야 할까?

1.

```
student_0 = 'asdf'
```

```
student_1 = 'qwerty'
```

```
student_2 = 'hiarc'
```

...

```
student_29 = 'OTL'
```

2.

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```

01

리스트 선언하기

(리스트 이름) = []

(리스트 이름) = [1, 2, 3, 'ㄱㄴㅇㄹㄹㄴㅇ', ... , 0]

1.

student_0 = 'asdf'

student_1 = 'qwerty'

student_2 = 'hiarc'

...

student_29 = 'OTL'

2.

student = ['asdf', 'querty', 'hiarc', ... , 'OTL']

이렇게 하나로 뭉뚱그리면 'hiarc'는 어떻게 찾죠?

1.

```
student_0 = 'asdf'
```

```
student_1 = 'qwerty'
```

```
student_2 = 'hiarc'
```

...

```
student_29 = 'OTL'
```

2.

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```




01

리스트 인덱싱

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```

0 1 2 ... 29

```
student[2] == 'hiarc' # True
```

01

리스트 인덱싱

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']  
          0       1       2     ...  29  
          -30    -29    -28    ...  -1
```

```
student[2] == 'hiarc' # True
```

```
student[-28] == 'hiarc' # True
```

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```

0	1	2	...	29
-30	-29	-28	...	-1

```
student[2] == 'hiarc' # True
```

```
student[-28] == 'hiarc' # True
```

```
student[30]
```

```
student[-31] # Error, Index out of range
```


01

리스트 인덱싱

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```

0	1	2	...	29
-30	-29	-28	...	-1

```
student[2] = 'GDG'
```



```
student = ['asdf', 'querty', 'GDG', ..., 'OTL']
```

01

리스트 슬라이싱

리스트인 채로 일부만 잘라서 쓰고 싶다면...

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']  
          0       1       2   ...  29  
          -30    -29    -28   ...  -1
```

```
student[0:3] == ['asdf', 'querty', 'hiarc'] # True
```

```
student[:3] == ['asdf', 'querty', 'hiarc'] # True
```

```
student[29:] == ['OTL'] # True
```

```
student[:] == ['asdf', 'querty', 'hiarc', ..., 'OTL'] # True
```


01

리스트 슬라이싱

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']  
          0       1       2     ...  29  
          -30     -29     -28   ...  -1
```

```
student[31:35] == [] # True
```

놀랍게도 오류가 나지 않는다!!!



01

연산자

만약 초급 스터디에서 3명이 기초 스터디로 수강 정정했다면?

```
student = ['asdf', 'querty', 'hiarc', ... , 'OTL' ]
```

```
noviceStudent = ['abc', 'def', 'ghi']
```

```
newStudent = student + noviceStudent
```

```
# ['asdf', 'querty', 'hiarc', ... , 'OTL', 'abc', 'def', 'ghi' ]
```

리스트끼리 덧셈이 가능하다.

01

연산자

덧셈의 함정

```
>>> a = [1, 2]
>>> b = [3, 4, 5]
>>> id(a), id(b)
(1354312109184, 1354311967872)
>>> a += b
>>> id(a)
1354312109184
>>> c = a + b
>>> id(c)
1354311974400
>>> a = a + b
>>> id(a)
1354312069824
```

+=와 +가 다르다?!

덧셈의 함정

$+=$ 는 기존 리스트에 피연산자 리스트를 덧붙이지만,
 $+$ 는 두 리스트를 이어 완전히 새로운 리스트를 만든다.

즉, 용도가 다르며 메모리 사용량과 소요 시간도 다르다.

01

연산자

```
1 import sys
2 from time import time
3 input = sys.stdin.readline
4 SIZE = 10**7
5 a = [0]*SIZE
6 b = [0]*SIZE
7
8 st = time()
9 a += b
10 ed = time()
11 print(ed-st)
12
13 a = [0]*SIZE
14 st = time()
15 a = a+b
16 ed = time()
17 print(ed-st)
18
19
```

main ×

⋮

C:\Users\이가은\PycharmProjects\SOLVE\venv\Scripts\python.exe C:\Users\이가은\PycharmProjects\SOLVE\main.py
0.0539700984954834
0.08699727058410645

덧셈의 함정

일반적으로 리스트에 요소를 추가할 때는 뒤에 배울 `.append()`를 사용.

+는 +=나 `.append()`에 비해 느리므로
크기가 큰 리스트를 연산할 때나 반복문 안에서는 사용을 자제하자!

리스트 곱셈

$A = [0] * 5$

$\rightarrow A = [0, 0, 0, 0, 0]$

주의사항: 반드시 1차원 리스트에서만 사용하도록 하자.

리스트 안에 다른 리스트가 존재할 경우 무서운 일이 일어날 수 있다.

(깊은복사, 얕은복사)

01

연산자

```
>>> a = [1, [1]]
>>> b = a*3
>>> b
[1, [1], 1, [1], 1, [1]]
>>> b[1][0] = 3
>>> b
[1, [3], 1, [3], 1, [3]]
```



01

리스트 순회하기

리스트 안의 원소를 차례대로 꺼내서 쓰고 싶어요.

01

리스트 순회하기

```
student = ['asdf', 'qwerty', 'hiarc', ..., 'OTL']
```

```
for handle in student:  
    print(handle)
```



```
asdf  
qwerty  
hiarc  
...  
OTL
```


01

리스트 순회하기

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```

```
for handle in student[:3]:  
    print(handle)
```



```
asdf  
querty  
hiarc
```

01

리스트 순회하기

```
student = ['asdf', 'qwerty', 'hiarc', ..., 'OTL']
```

```
for i in range(30):  
    print(student[i])
```



asdf

qwerty

hiarc

...

OTL



01

리스트 순회하기

한 줄에 예쁘게 출력하려면 어떻게 하죠?

01

리스트 순회하기

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```

```
for i in range(30):  
    print(student[i], end=' ')
```



asdf querty hiarc ... OTL

Emphasize

end를 설정하지 않은 기본값은 '\n' 이다.
따라서 end를 임의로 설정하면 루프가 끝난 뒤 print()를 한번 해줘야 다음 출력부터는 다른 줄에 출력되니 주의.

01

리스트 순회하기

```
student = ['asdf', 'querty', 'hiarc', ..., 'OTL']
```

```
print(*student)
```



```
asdf querty hiarc ... OTL
```

Emphasize

반복 가능한 객체 앞에 '*' 를 붙이면 알아서 띄어쓰기로 구분해 예쁘게 출력해준다.



예제: 과제 안 내신 분..? (#5597)

같이 풀어봅시다.

예제: 과제 안 내신 분..? (#5597)

```
import sys
input = sys.stdin.readline

arr = [0]*31
for _ in range(28):
    arr[int(input().strip())] = 1

for i in range(1, 31):
    if not arr[i]:
        print(i)
```



01

리스트 편집하기

만들어진 리스트에 원소를 추가할래요.

01

리스트 편집하기

append 메소드 사용하기

`arr.append(6)`

`arr = [0, 1, 2, 3, 4, 5]`

→

`arr = [0, 1, 2, 3, 4, 5, 6]`

`(추가할 리스트).append(추가할 원소)`

→ 리스트 맨 뒤에 원소 추가



01

리스트 편집하기

메소드?



01

리스트 편집하기

메소드?

객체 내에 정의된 함수

(객체).(메소드명) (매개변수) 로 사용

메소드?

`input().strip()` 도
`input()` 으로 받은 `string` 타입에 정의되어 있는
`.strip()`이라는 메소드를 사용하는 것이다.

실행될 때는 왼쪽부터 차근차근, 괄호 안쪽 우선
`list(map(int, input().split()))` 은
함수와 메소드를 같이 사용하는 예제이다.

`list(map(int, input().split()))`

1. `input()`으로 문자열 입력 -> "1 2 3\n"
2. `split()`으로 분할 -> ['1', '2', '3']
3. `map()`으로 `int()`적용 -> map obj (1, 2, 3)
4. `list()`로 형변환 -> [1, 2, 3]

01

리스트 편집하기

pop 메소드 사용하기

`arr.pop()`

`arr = [0, 1, 2, 3, 4, 5]`

→

`arr = [0, 1, 2, 3, 4]`

`(제거할 리스트).pop(제거할 인덱스)`

-> 리스트에서 제거할 인덱스에 있는 원소를 삭제한 후 값을 반환
(기본값 -1)

pop 메소드 사용하기

`A = arr.pop() == 5`

`arr = [0, 1, 2, 3, 4, 5]` $\rightarrow$ `arr = [0, 1, 2, 3, 4]`

(제거할 리스트).pop(제거할 인덱스)

-> 리스트에서 제거할 인덱스에 있는 원소를 삭제한 후 값을 반환
따라서 제거 후 A는 원래 arr의 마지막 원소였던 5가 됨



예제: X보다 작은 수 (#10871)

예제: X보다 작은 수 (#10871)

```
import sys
input = sys.stdin.readline

N, X = map(int, input().split())
A = list(map(int, input().split()))
res = []
for i in A:
    if i < X:
        res.append(i)

print(*res)
```

01

리스트 편집하기

sort 메소드 사용하기

`arr.sort()`

`arr = [1, 3, 2, 0, 4]`

→

`arr = [0, 1, 2, 3, 4]`

`(정렬할 리스트).sort()`

→ 리스트의 원소를 오름차순으로 정렬

01

리스트 편집하기

sort 메소드 사용하기

`arr.sort(reverse=True)`

`arr = [1, 3, 2, 0, 4]` → `arr = [4, 3, 2, 1, 0]`

`(정렬할 리스트).sort(reverse=True)`

→ 리스트의 원소를 내림차순으로 정렬

그럼 sorted()는 뭐지..?

arr = [1, 3, 2, 0, 4]

A = sorted(arr)



arr = [1, 3, 2, 0, 4]

A = [0, 1, 2, 3, 4]

sorted(정렬할 리스트)

리스트 안의 원소를 정렬한 새로운 리스트를 반환함.

원본 리스트에는 영향 x

```
arr = [i for i in range(5)]      = [0, 1, 2, 3, 4]
```

```
arr = [i*2 for i in range(5)]    = [0, 2, 4, 6, 8]
```

[추가할 원소(식) for ...]

-> 반복문을 수행하면서 리스트에 원소 추가

01

리스트 선언하기-2

```
arr = [i for i in range(5)] == arr = []  
                               for i in range(5):  
                                 arr.append(i)
```

압축해서 편리하게 사용할 수 있다.

01

리스트 선언하기-2

이런건 어떤가요?

```
arr = [[0]*3 for _ in range(3)] == arr = []  
                                     for _ in range(3):  
                                         arr.append([0]*3)
```

arr = ???

이런건 어떤가요?

```
arr = [[0]*3 for _ in range(3)] == arr = []  
                                     for _ in range(3):  
                                         arr.append([0]*3)
```

```
arr = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]  
이것이 2차원 리스트!
```


이런건 어떤가요?

```
arr = [[0]*3 for _ in range(3)] == arr = []  
                                     for _ in range(3):  
                                         arr.append([0]*3)
```

```
arr = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]  
이것이 2차원 리스트!
```

01

리스트 선언하기-2

arr = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]

행/열	0	1	2
0	0	0	0
1	0	0	0
2	0	0	0

arr[행][열]

arr[1][2] = 3

01

리스트 선언하기-2

arr = [[0, 0, 0], [0, 0, 3], [0, 0, 0]]

행/열	0	1	2
0	0	0	0
1	0	0	3
2	0	0	0

arr[행][열]

arr[1][2] = 3

물론, 진짜 행렬과는 달리
리스트에서는 행끼리 길이가 다를 수 있다.



예제: 행렬 덧셈 (#2738)

같이 풀어봅시다.

예제: 행렬 덧셈 (#2738)

```
import sys
input = sys.stdin.readline

N, M = map(int, input().split())

matrix_1, matrix_2 = [], []
res = [[0]*M for _ in range(N)]

for _ in range(N):
    row = list(map(int, input().split()))
    matrix_1.append(row)

for _ in range(N):
    row = list(map(int, input().split()))
    matrix_2.append(row)

for i in range(N):
    for j in range(M):
        res[i][j] = matrix_1[i][j] + matrix_2[i][j]

for i in range(N):
    print(*res[i])
```


테코
퓨얼

—



튜플은 수정할 수 없는 리스트입니다.

선언하는 법:

A = (1, 2, 3) # 리스트와 달리 소괄호

b = (1,) # 원소가 한개여도 쉼표 필요

C = 1, 2, 3 # 때에 따라 괄호는 생략

튜플은 수정할 수 없는 리스트입니다.
그래도 +와 +=는 여전히 사용 가능합니다.
이번에는 +=도 +와 하는 일이 같습니다.
왜냐하면 튜플은 덧붙일 수 없고
두 개를 더하려면
그냥 새로 하나 만드는 수밖에 없습니다.
근데 사실 튜플끼리 +는 쓸 일이 없긴 합니다.

그래서 보통 리스트와는 쓰임새가 다릅니다.
리스트가 값들을 넣었다 뺐다 하는 박스라면
튜플은 박스에 들어가는 물건(인데 조금 크기가 큰)입니다.
무슨 말인지는 문제를 풀어가면서 차차 깨닫게 되실 겁니다.

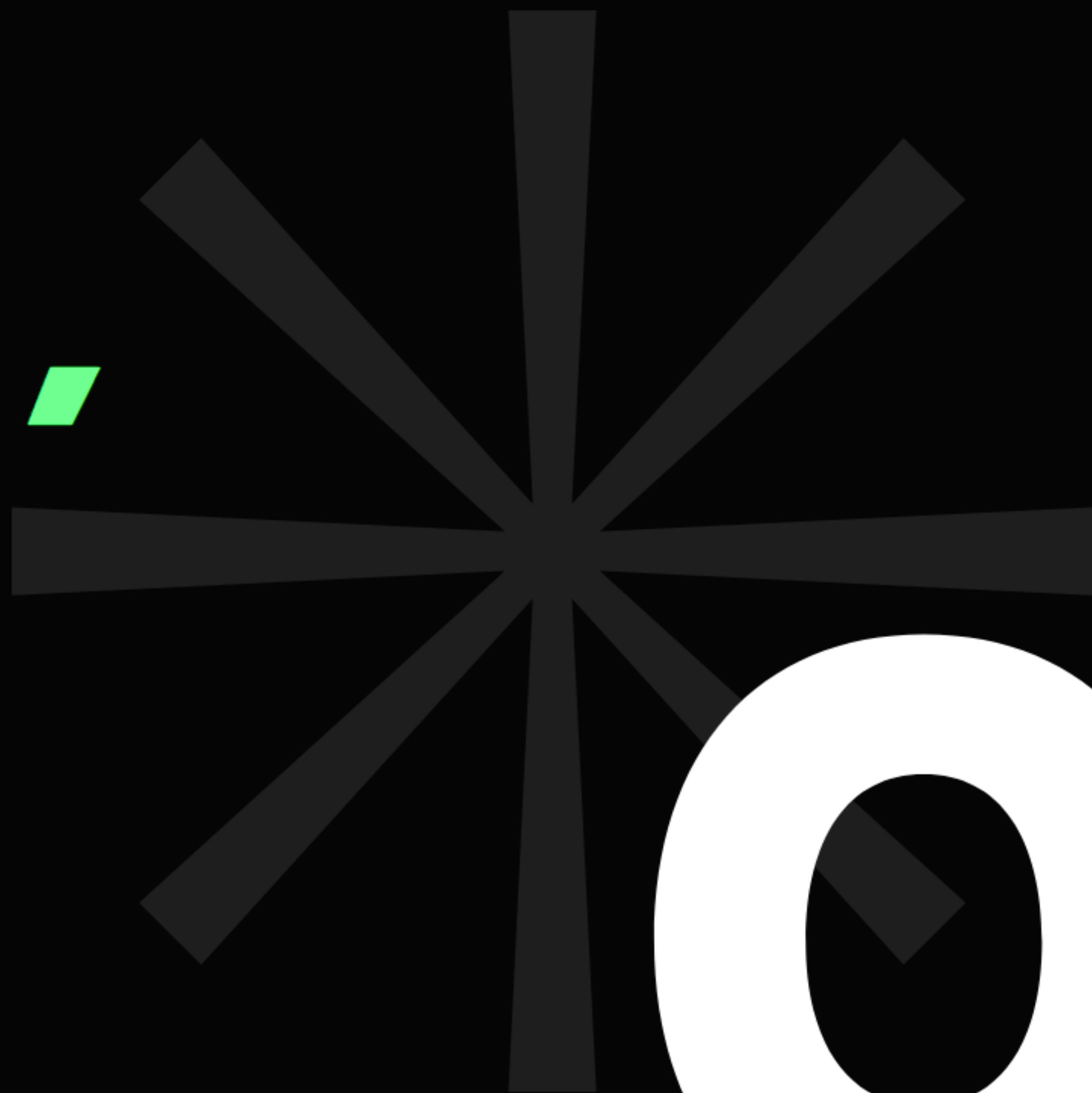
튜플은 수정이 불가능하기 때문에
리스트보다 메모리 면에서 효율적입니다.
리스트는 추가/삭제를 위해 여유 공간을 두기 때문입니다.
입력이 10만개 들어와서 원소 10만개를 만들어야 한다면
리스트 10만개를 만드는 것 보다 튜플 10만개를 만드는 게
더 이상적입니다.

그래서 결론은
인덱싱(대괄호 문법 [n]), 슬라이싱, for문 순회 등
모든 것이 리스트와 똑같이 때문에...



딕셔너리,
세트

—



03

딕셔너리는 원소를 키:값 형태로 저장할 수 있다.
저장한 원소를 인덱스 순서가 아닌 지정한 이름(key)으로 부르고 싶을 때 사용.

세트는 집합이다.
중복된 값을 저장하면 한개만 저장된다.
원소간에는 순서가 없다.
또한 교집합, 합집합 등의 연산이 가능하다.

딕셔너리 선언

`dic = {}` 또는 `dic = {'a':1, 'b':2}` 또는 `dic = dict()`

세트 선언

`s = set()` 또는 `s = {1, 2, 3}`

세트는 왜 그냥 선언이 안 되죠??

-> 딕셔너리와 똑같이 중괄호를 쓰기 때문에 중괄호만 쓰면 딕셔너리가 됩니다.

딕셔너리에 요소 추가하기

```
dic[키값] = 값
```

```
dic[키값] = []
```

딕셔너리 요소 접근하기

`dic[키값]` → 키값에 할당된 값 반환

`dic[키값] = []` 일 경우

`dic[키값].append(a)...` 등 가능

인덱싱은 2차원 리스트처럼 `dic[키값][n]`

딕셔너리 요소 순회하기

for **key** in dic: # 키값만 순회

for **value** in dic.values(): # 값만 순회

for **key, value** in dic.items(): # 키, 값 순회

.items로 알아보는 놀라운 사실:

.items() 메소드는 딕셔너리 안에 있는 키:값을 각각 튜플(키, 값) 으로서 순회할 수 있게 해준다.

즉, 튜플 안에 리스트가 들어갈 수도 있다.

또한 for문 안에서 순회하는 값이 튜플이나 리스트일 경우 변수 한꺼번에 선언하기와 마찬가지로 여러 이름에 나눠서 저장할 수 있다!! 이 사실은 유용하니 기억해두자.

세트 연산하기

📌 (2) 세트 수정 관련 메서드

메서드	설명
<code>set.add(x)</code>	요소 <code>x</code> 를 추가
<code>set.remove(x)</code>	요소 <code>x</code> 를 제거 (없는 요소 삭제 시 오류 발생)
<code>set.discard(x)</code>	요소 <code>x</code> 를 제거 (없는 요소 삭제해도 오류 없음)
<code>set.pop()</code>	임의의 요소를 제거하고 반환
<code>set.clear()</code>	모든 요소를 제거

세트에 쓸 수 있는 연산자와 메소드를 알려줘

파이썬의 **세트(set)**는 중복을 허용하지 않고, 순서가 없는 자료형이야.

세트에서는 여러 가지 연산자와 메서드를 사용할 수 있어! 🚀

◆ 1. 세트에서 사용 가능한 연산자

세트에서는 집합 연산자를 활용할 수 있어.

연산자	메서드	설명
<code> </code>	-	<code>union()</code>
<code>&</code>	<code>intersection()</code>	교집합 (두 집합 모두 포함된 요소)
<code>-</code>	<code>difference()</code>	차집합 (첫 번째 집합에만 있는 요소)
<code>^</code>	<code>symmetric_difference()</code>	대칭 차집합 (한쪽에만 존재하는 요소)
<code><=</code>	<code>issubset()</code>	부분집합 여부 확인 (A가 B의 부분집합인지)
<code>>=</code>	<code>issuperset()</code>	상위집합 여부 확인 (A가 B를 포함하는지)
<code>==</code>	-	두 집합이 동일한지 확인
<code>!=</code>	-	두 집합이 다른지 확인

방금 진심인가...? 라고 생각했나요?

농담입니다.

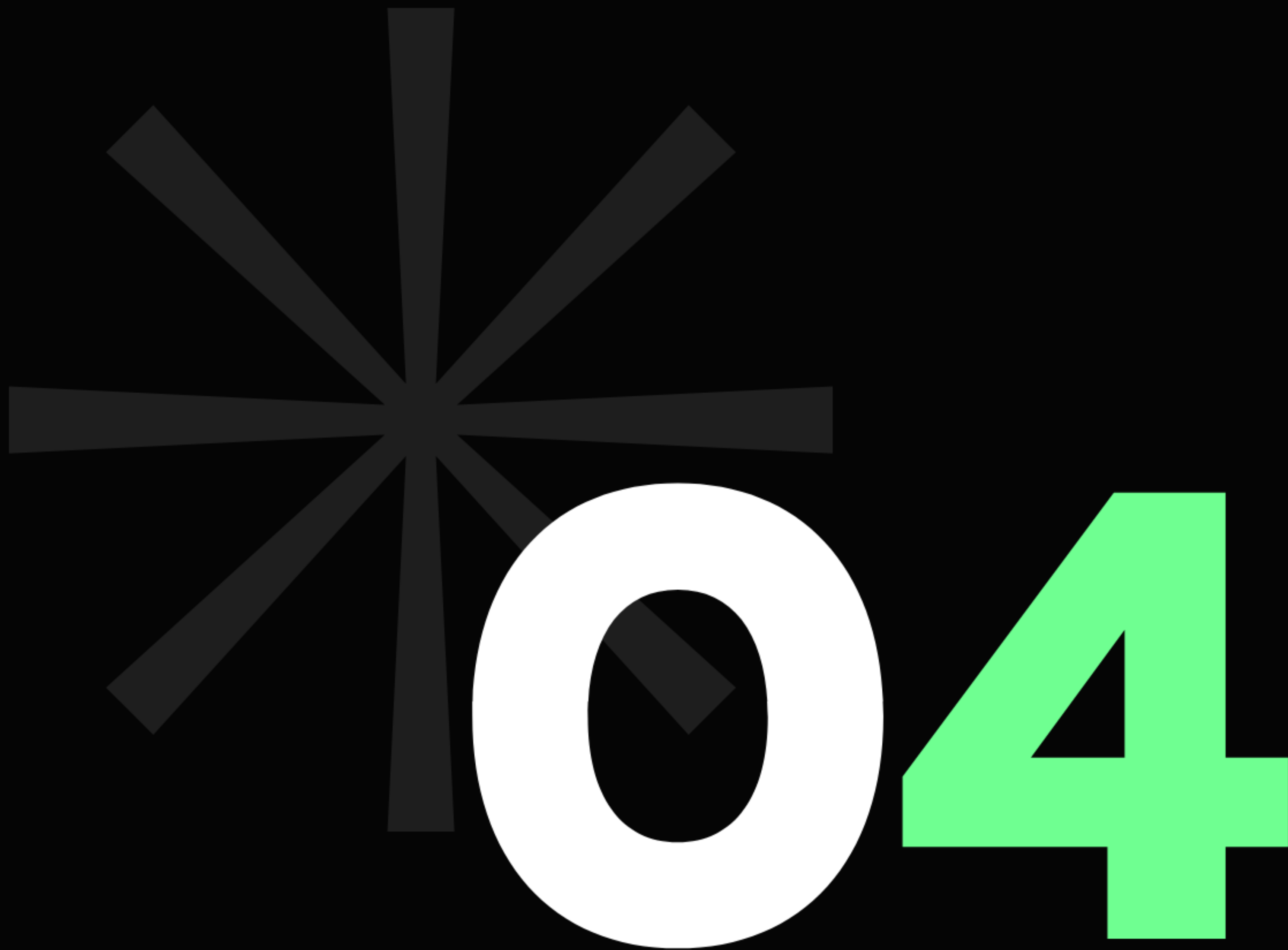
딕셔너리와 세트는 좋은 기능을 제공하긴 하지만
실전에서 문제를 풀면서 사용하기 쉽지 않습니다.
대부분의 알고리즘은 이 둘 없이 리스트, 튜플만으로 잘 구현되어 있습니다.

딕셔너리까지는 가끔 쓸만도 한데 `set()`는 쓰는걸 진짜 못봤습니다.
그러니 이 둘은 대충 흘려들으셔도 됩니다.



함수

—



함수(서브루틴)는
특정한 매개변수를 제공하면 미리 설정한 작업 후에
리턴값을 반환합니다.
수학처럼 input과 output이 있습니다.
리턴 없이 작업만 할 수도 있습니다.

함수 없이도 이론상 프로그램을 만들 수는 있습니다.

하지만 없으면 최악의 효율이 나옵니다.

또한 재귀(recursion) 구조는 함수 없이 구현하기 매우 힘듭니다.

함수의 기본 구조

```
def (함수이름) (매개변수1, ..., 매개변수N):
```

```
    ...
```

```
    return (값), 리턴이 없을 경우 None
```

```
ret = (함수이름) (변수1, ..., 변수N) # 설정한 리턴값이 할당됨.
```

```
(함수이름) (변수1, ..., 변수N) # 함수 실행 후 리턴값은 버림.
```

리스트의 각 원소에 1을 더하는 함수

```
def increase(lst):  
    for i in range(len(lst)):  
        lst[i] += 1  
  
    return
```

를 만들었다고 생각해 봅시다.

1.

```
a, b, c = [1, 2, 3], [4, 5, 6], [7, 8, 9]
```

```
for i in range(3):
```

```
    a[i] += 1
```

```
for i in range(3):
```

```
    b[i] += 1
```

```
for i in range(3):
```

```
    c[i] += 1
```

2.

```
a, b, c = [1, 2, 3], [4, 5, 6], [7, 8, 9]
```

```
increase(a)
```

```
increase(b)
```

```
increase(c)
```

함수를 왜 쓰나요?

1.

```
a, b, c = [1, 2, 3], [4, 5, 6], [7, 8, 9]
```

```
for i in range(3):
```

```
    a[i] += 1
```

```
for i in range(3):
```

```
    b[i] += 1
```

```
for i in range(3):
```

```
    c[i] += 1
```

2.

```
a, b, c = [1, 2, 3], [4, 5, 6], [7, 8, 9]
```

```
increase(a)
```

```
increase(b)
```

```
increase(c)
```

당연히 2가 읽는 쪽도 쓰는 쪽도 편하겠죠?

리스트의 길이를 재서 참 또는 거짓을 반환하는 함수

```
def check(lst, size): # lst: list, size: int
    if len(lst) > size:
        return True
    return False
```

를 만들었다고 생각해 봅시다.

리스트의 길이를 재서 참 또는 거짓을 반환하는 함수

```
def check(lst, size): # lst: list, size: int
    if len(lst) > size:
        return True
    return False
```

```
A = [1, 2, 3]
B = check(A, 2) # False

if check(A, 3):
    ...
```

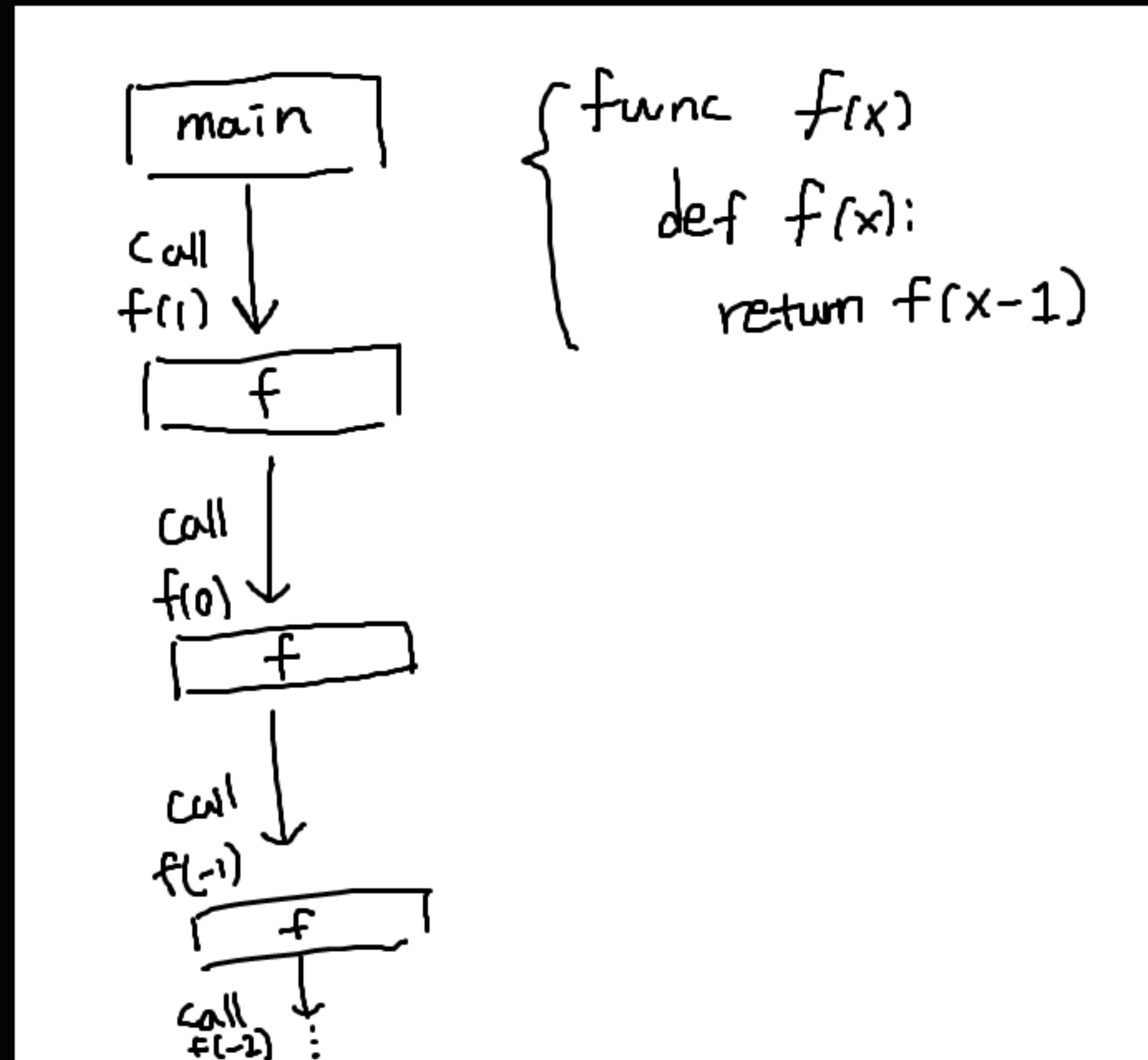
함수가 호출되면, 하던 작업을 멈추고 호출된 함수부터 수행함.

재귀(recursion)

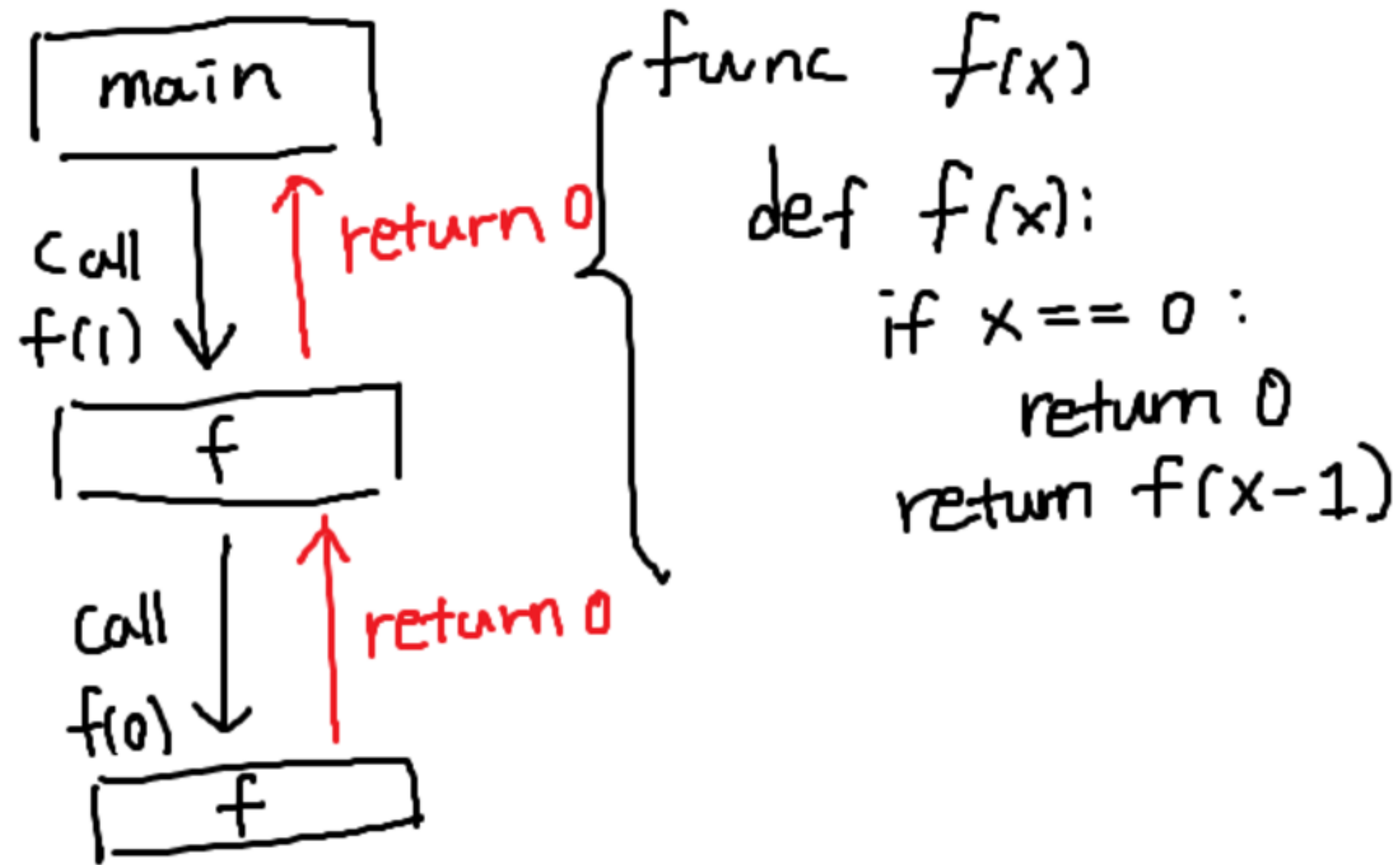
함수 내에서 자기 자신을 재호출하는 경우 그 함수를 **재귀함수**라고 함.

재귀함수에는 반드시 탈출조건이 필요하다.

그렇지 않으면 끝없이 자신을 재호출하면서 결국에는 프로그램이 터진다.



재호출을 하지 않고 값을 리턴하는 탈출조건이 있어야
프로그램을 정상적으로 작동시킬 수 있다!



재귀함수로 팩토리얼을 계산하려면?

$$N! = N * (N-1)!$$

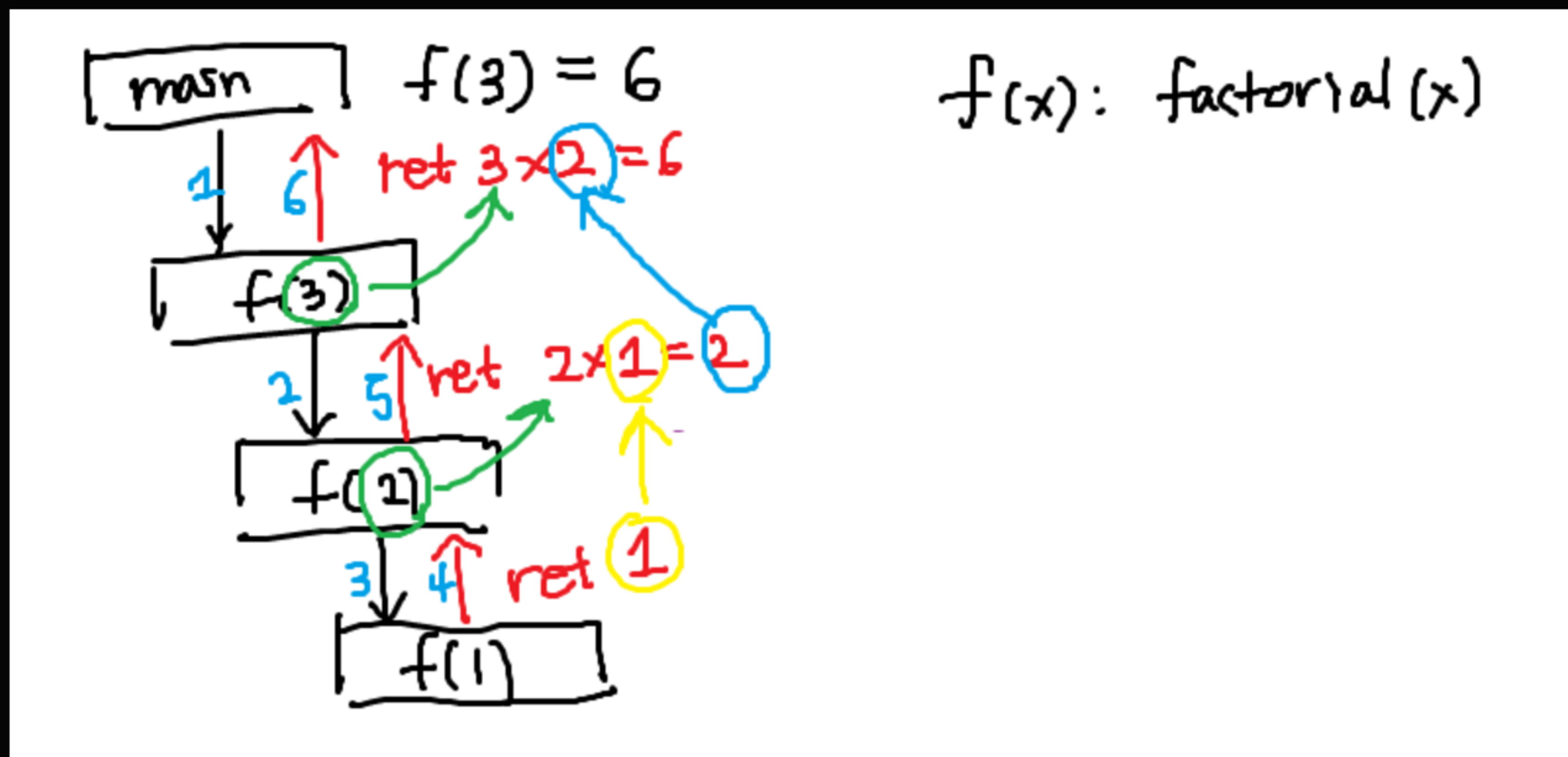
$$\text{factorial}(N) = N * \text{factorial}(N-1)$$

이런 것을 점화식 또는 재귀식(recurrence relation)이라고 합니다.

재귀함수로 팩토리얼을 계산하려면? ($n > 0$)

```
def factorial(n):  
    if n == 1: # 탈출조건, 첫 항  
        return 1  
    return n*factorial(n-1)
```

재귀함수로 팩토리얼을 계산하려면? ($n > 0$)



04

재귀

```
1 import sys
2 input = sys.stdin.readline
3
4
5     2 개의 사용 위치
6 def factorial(n):
7     if n == 0:
8         return 1
9     else:
10        return n*factorial(n-1)
11
12 factorial(998)
```

main ×

⋮

```
    return n*factorial(n-1)
```

[Previous line repeated 995 more times]

File "C:\Users\이가은\PycharmProjects\SOLVE\main.py", line 6, in factorial

```
    if n == 0:
```

RecursionError: maximum recursion depth exceeded in comparison

종료 코드 1(으)로 완료된 프로세스

흠..근데 왜 998! 은 안 되나요...

파이썬의 기본 재귀 깊이는 1000
그 이상 호출하려면 수동으로 제한을 풀어줘야 함.

`sys.setrecursionlimit(깊이 제한)`

```
import sys
 = sys.stdin.readline
sys.setrecursionlimit(10**5)
```

저도 자주 까먹어서 한번씩 틀려요...

재귀함수로 할 수 있는 것들:

팩토리얼

피보나치 수열

그래프 탐색(dfs)

다이나믹 프로그래밍

분할정복

등등등... 점화식이 있는 알고리즘



04

재귀

예제: 재귀함수가 뭔가요?
(#17478)

지문이 너무 길어지니 언더바는 무시하고 파트별로 나눠봅시다.

H : 어느 한 컴퓨터공학과 학생이...

A : "재귀함수가 뭔가요?"

B : "잘 들어보게. 옛날옛날 한 산 꼭대기에 이세상 모든 지식을 통달한 선인이 있었어....(생략)

C : 라고 답변하였지.

D : "재귀함수는 자기 자신을 호출하는 함수라네"

$N == 2$

H

A

B

___A

___B

_____A

_____D

_____C

___C

C

A, B를 반복해서 출력하다가
재호출 횟수가 $N=2$ 가 되면 탈출조건 실행
→ A, D, C를 출력함.

이제부터 함수가 종료(return)되기 시작
종료될 때는 C를 출력.

언더바는 해당 줄을 출력할 때의 재귀 횟수를 뜻함.
문자열도 곱셈이 가능함을 잊지 말자.

04

재귀

$N == 2$

H

A

B

__A

__B

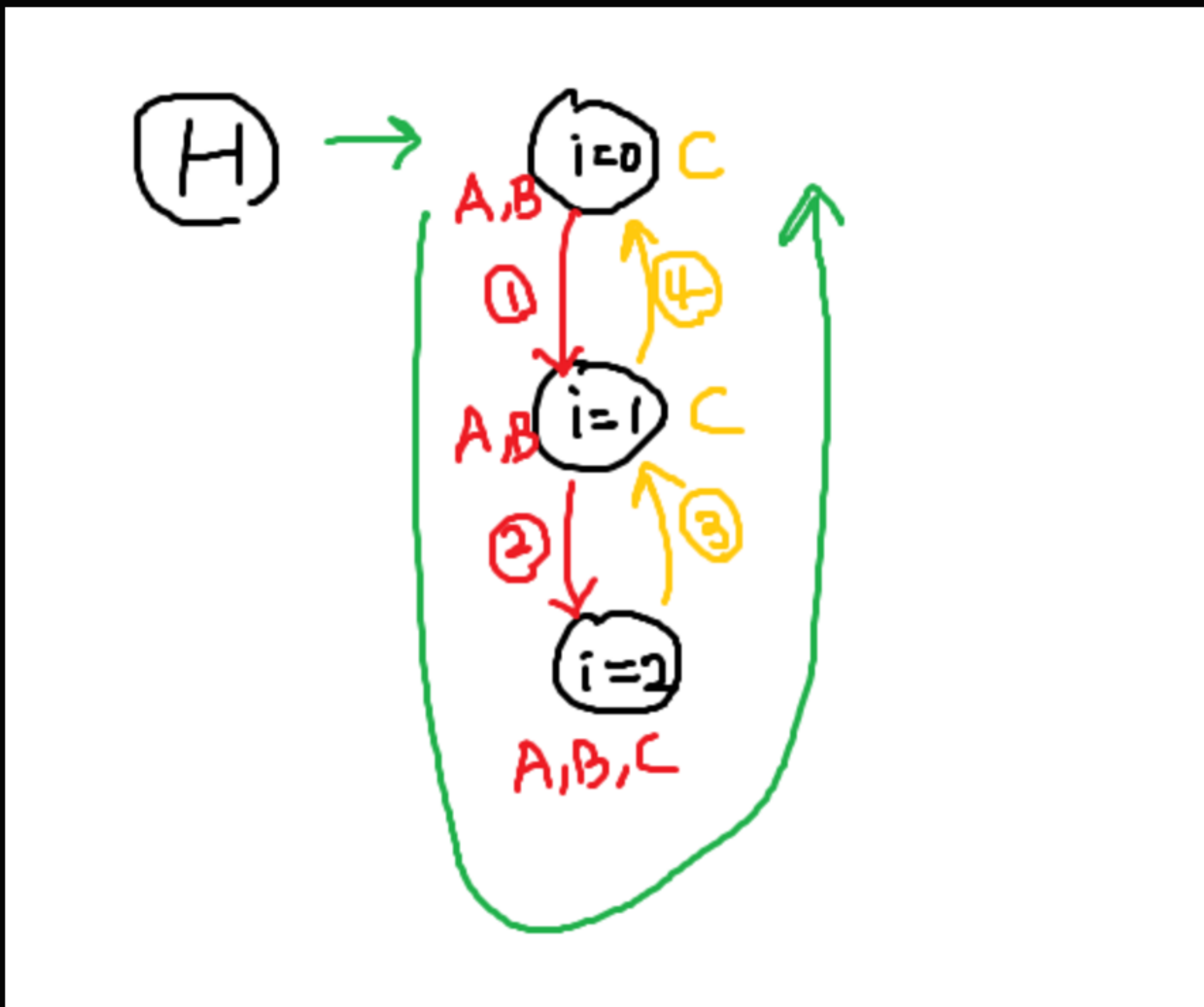
____A

____D

____C

__C

C



그렇다면 재귀호출에 필요한 정보는

1. 현재까지 재귀한 횟수
2. 탈출 조건

이 둘을 매개변수로 넘기면서 재귀호출을 진행해보자.

```
def recursion(i, N) # (재귀횟수, 탈출조건)
```

04

재귀

```
import sys
 = sys.stdin.readline
```

```
N = int(input().strip())
```

7 개의 사용 위치

```
def line(l, string):
    print("____"*l+string)
```

2 개의 사용 위치

```
def recur(i, end):
    line(i, string: "\재귀함수가 뭔가요?\")
    if i == end:
        line(i, string: "\재귀함수는 자기 자신을 호출하는 함수라네")
        line(i, string: "라고 답변하였지.")
        return
    line(i, string: "\잘 들어보게. 옛날옛날 한 산 꼭대기에 이세상 모든 지식을 통달한 선인이 있었어.")
    line(i, string: "마을 사람들은 모두 그 선인에게 수많은 질문을 했고, 모두 지혜롭게 대답해 주었지.")
    line(i, string: "그의 답은 대부분 옳았다고 하네. 그런데 어느 날, 그 선인에게 한 선비가 찾아와서 물었어.\")
    recur(i+1, end)
    line(i, string: "라고 답변하였지.")

print("어느 한 컴퓨터공학과 학생이 유명한 교수님을 찾아가 물었다.")
recur(i: 0, N)
```


익명 함수(lambda)

함수를 따로 선언하지 않고 사용하는 1회용 함수

lambda (매개변수): (수식)

익명 함수(lambda)

함수를 따로 선언하지 않고 사용하는 1회용 함수

```
(lambda x, y: x+y)(1, 2) # 3
```

그냥 $x+y$ 를 하면 되는거 아니가요...?

map(함수, 반복 가능한 객체 (리스트, 튜플 등)) 을 기억하시나요?

매개변수로 함수가 들어가는 경우 람다를 사용하는 경우가 많습니다.

(반복 가능한 객체).sort(**key = lambda ...**)

정렬을 진행하면서 값을 비교할 때 각 원소에 key 함수를 적용한 채로 비교

```
lst = [(1, 3), (2, 2), (3, 1)]
```

```
lst.sort() # [(1, 3), (2, 2), (3, 1)]
```

```
lst.sort(key = lambda x: x[1]) # [(3, 1), (2, 2), (1, 3)]
```

단, `sort()`의 경우 기본적으로는 사전식으로 정렬되지만
`key`를 설정할 경우 동일 우선순위일 때는 기존의 순서를 유지함.


```
lst = [(1, 3), (2, 2), (3, 1)]
```

```
lst.sort() # [(1, 3), (2, 2), (3, 1)]
```

```
lst.sort(key = lambda x: x[1]) # [(3, 1), (2, 2), (1, 3)]
```

더 복잡한 정렬 기준을 원할 경우

functools에 있는 `cmp_to_key()`를 사용해야 함.

기초 스터디에서 다룰만한 내용은 아니므로 궁금한 사람만 찾아보길 바람.



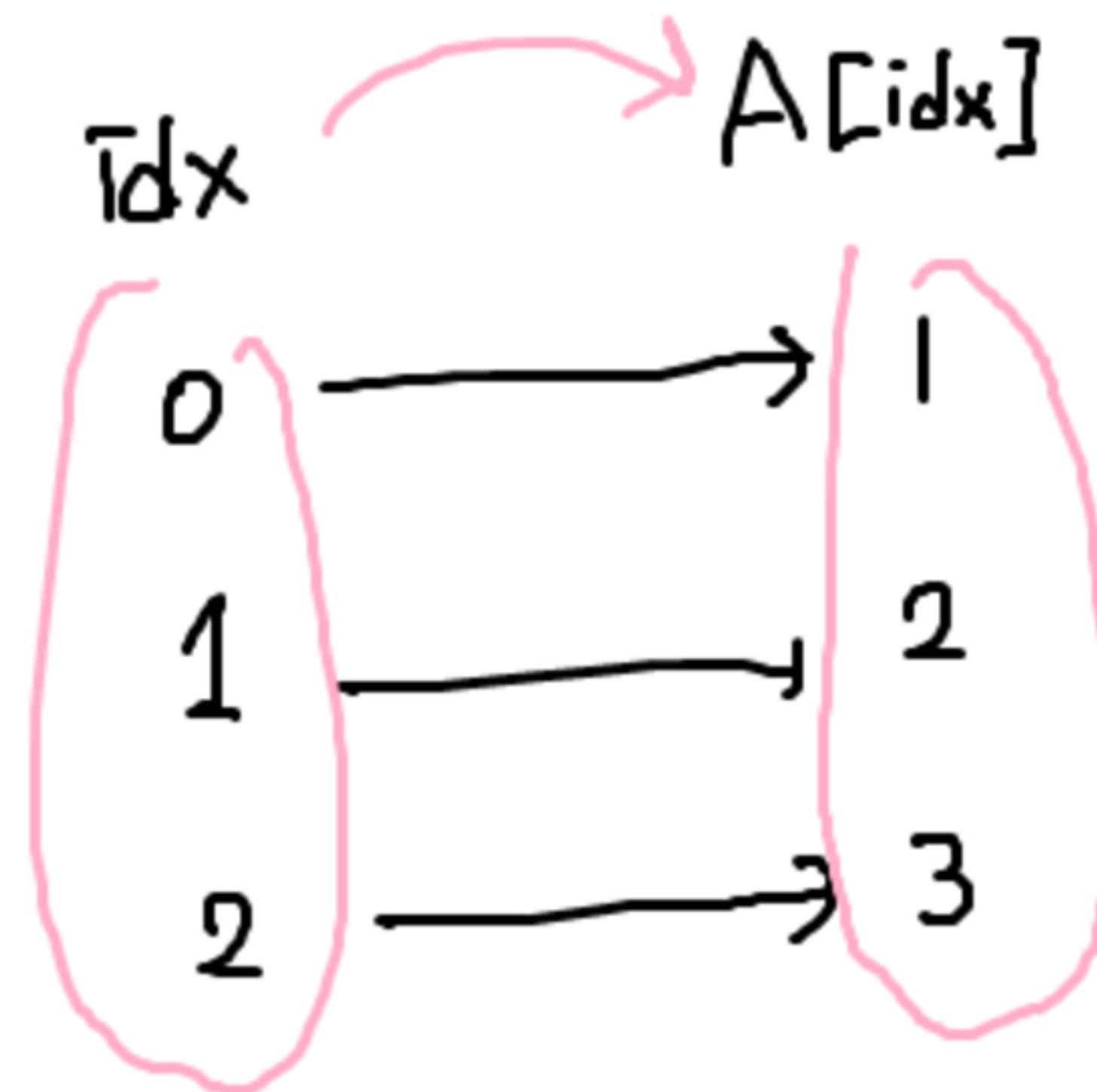
04

익명 함수

예제: 수열 정렬 (#1015)

배열 $A[x]$ 를 함수로 생각해보자.

$A = [1, 2, 3]$ 일 때
인덱스와 각 인덱스에 매칭되는
 A 의 원소로
함수를 구성할 수 있다!



$$B[P[i]] = A[i]$$

이 식을 변형해서 구해야 하는 P 를 꺼내 보자.

각 변에 B' (B 의 역함수)를 취해 주면

$$P[i] = B'[A[i]]$$

A 는 입력으로 들어오고, B 는 A 와 원소는 같지만 정렬된 배열이다.

즉 A 를 정렬하면 B 가 된다.

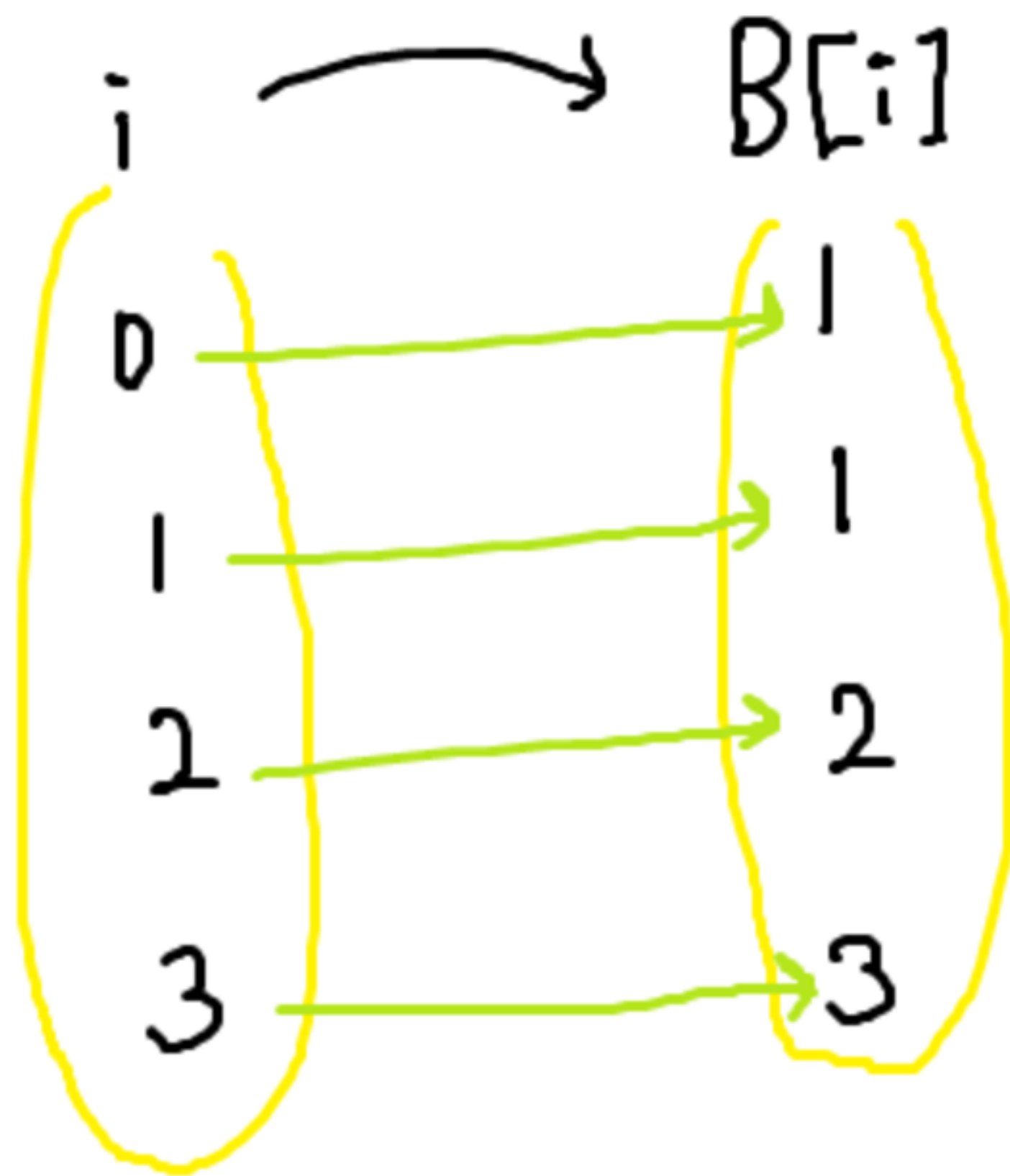
구해야 하는 수열 P 는 B' 와 A 의 합성함수임을 알 수 있다.
 B 와 A 를 모두 알고 있으므로 남은건 어떻게 B 에서 B' 를 만드느냐 인데...

우선 A를 정렬해보자.

$A = [2, 1, 3, 1]$ 을 정렬하면 $[1, 1, 2, 3]$

그런데 문제가 있다.

B에 중복되는 원소가 있어서
이대로 역함수를 만들면
곤란해질 것 같다..

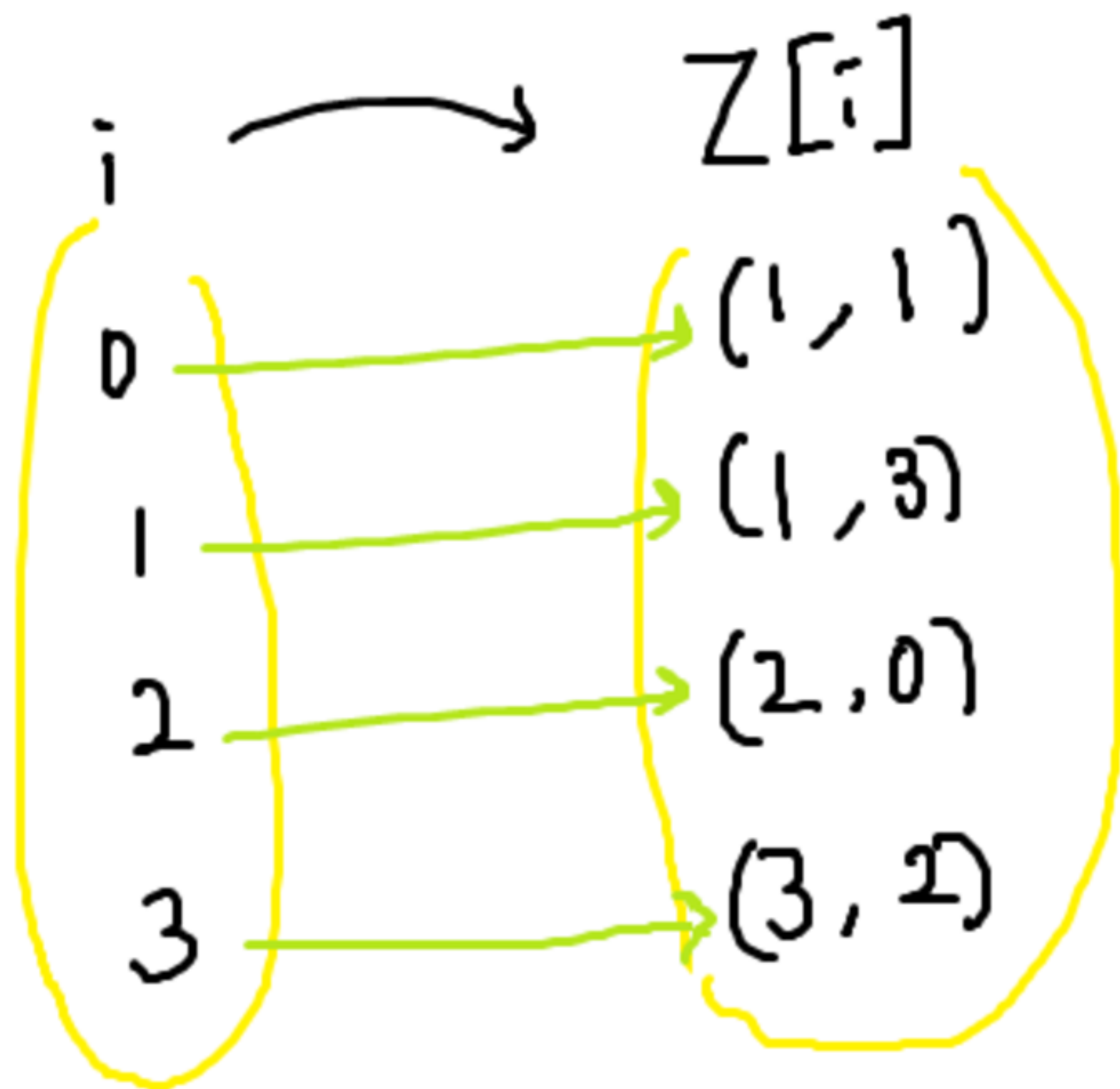


이렇게 해보자!
원소와 인덱스가 튜플로 붙어있는
새로운 배열 Z를 만들자.

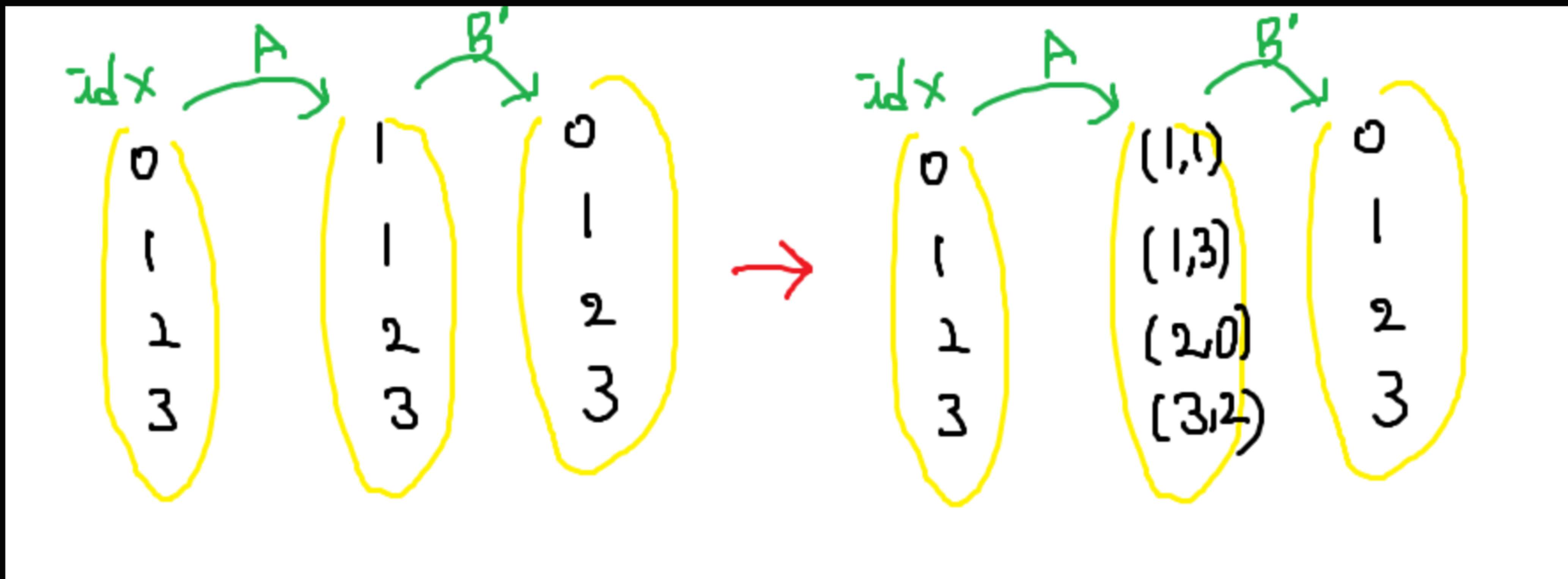
$Z = [(A[i], i) \text{ for } i \text{ in range}(N)]$

이 다음 $Z.sort()$ 를 실행하면
 $A[i]$ 순으로 정렬되지만
원래의 인덱스가 저장되어 있다.

$A = [2, 1, 3, 1], B = [1, 1, 2, 3]$



$$A = [2, 1, 3, 1], B = [1, 1, 2, 3]$$

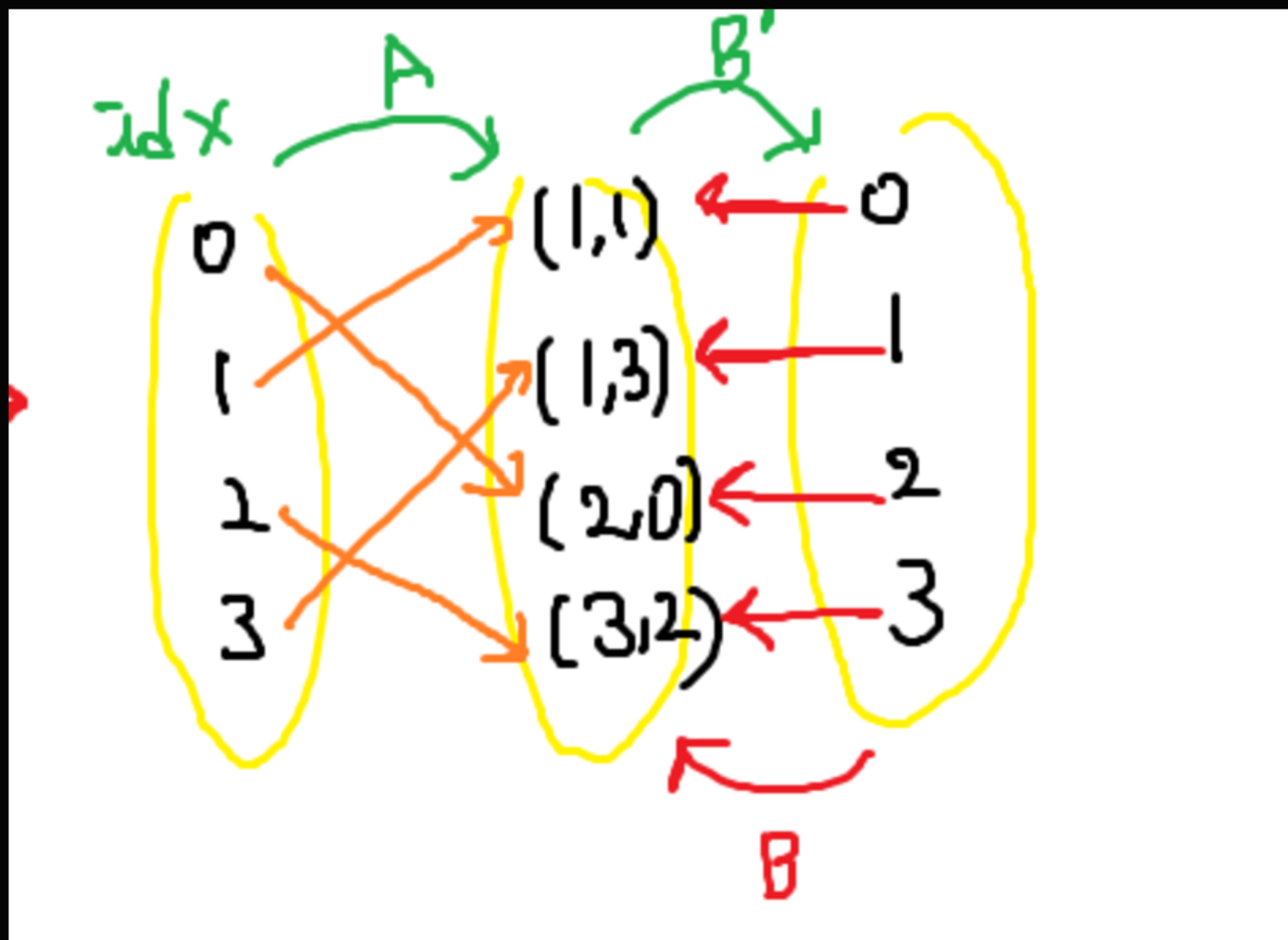


이렇게 하면 최종적으로 오른쪽의 대응관계가 만들어질 것이다.

04

익명 함수

$$A = [2, 1, 3, 1], B = [1, 1, 2, 3]$$

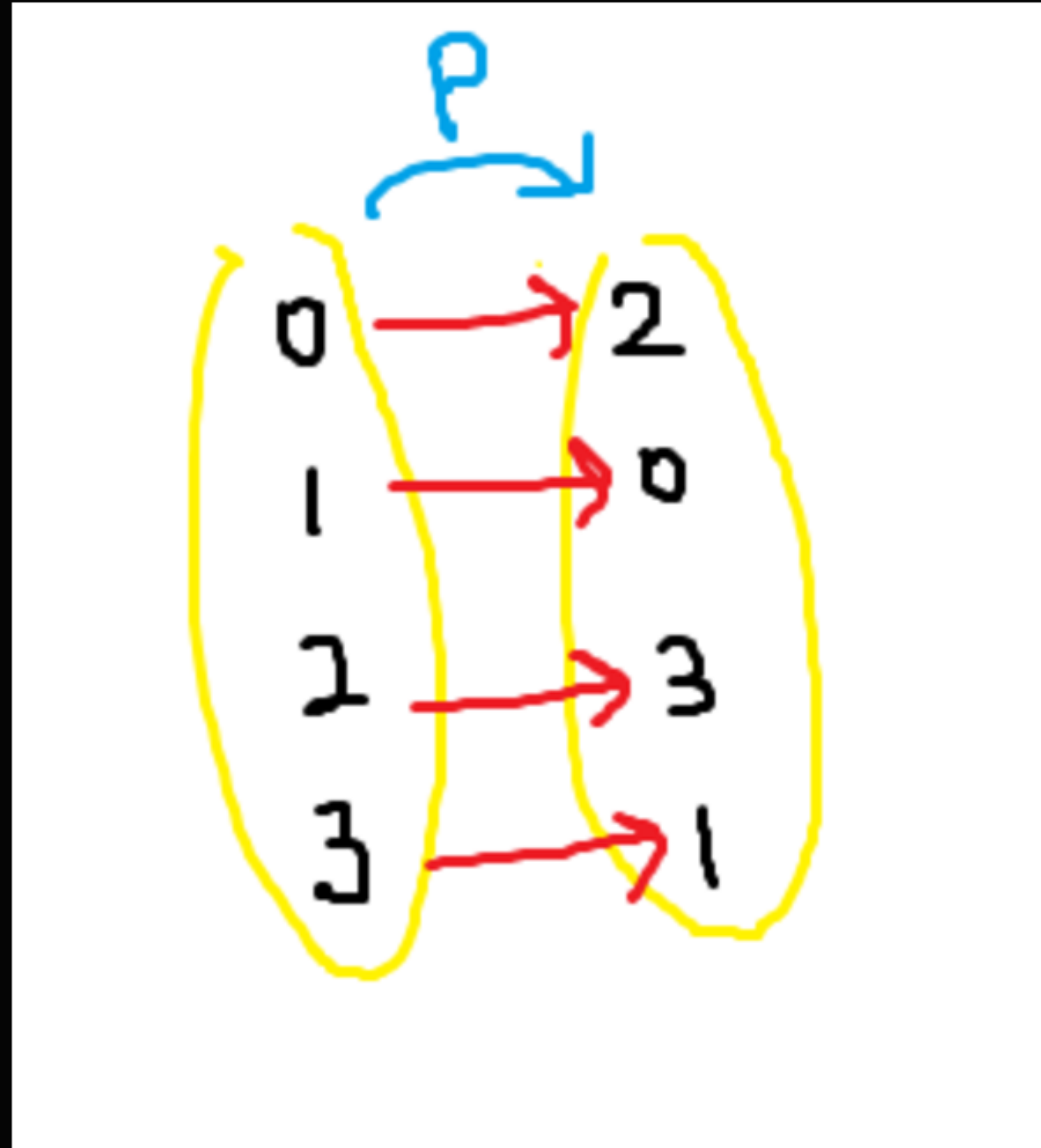


여기서 이미 알고있는 A와 B를 이용해 관계를 표시해주자.

04

익명 함수

$$A = [2, 1, 3, 1], B = [1, 1, 2, 3]$$

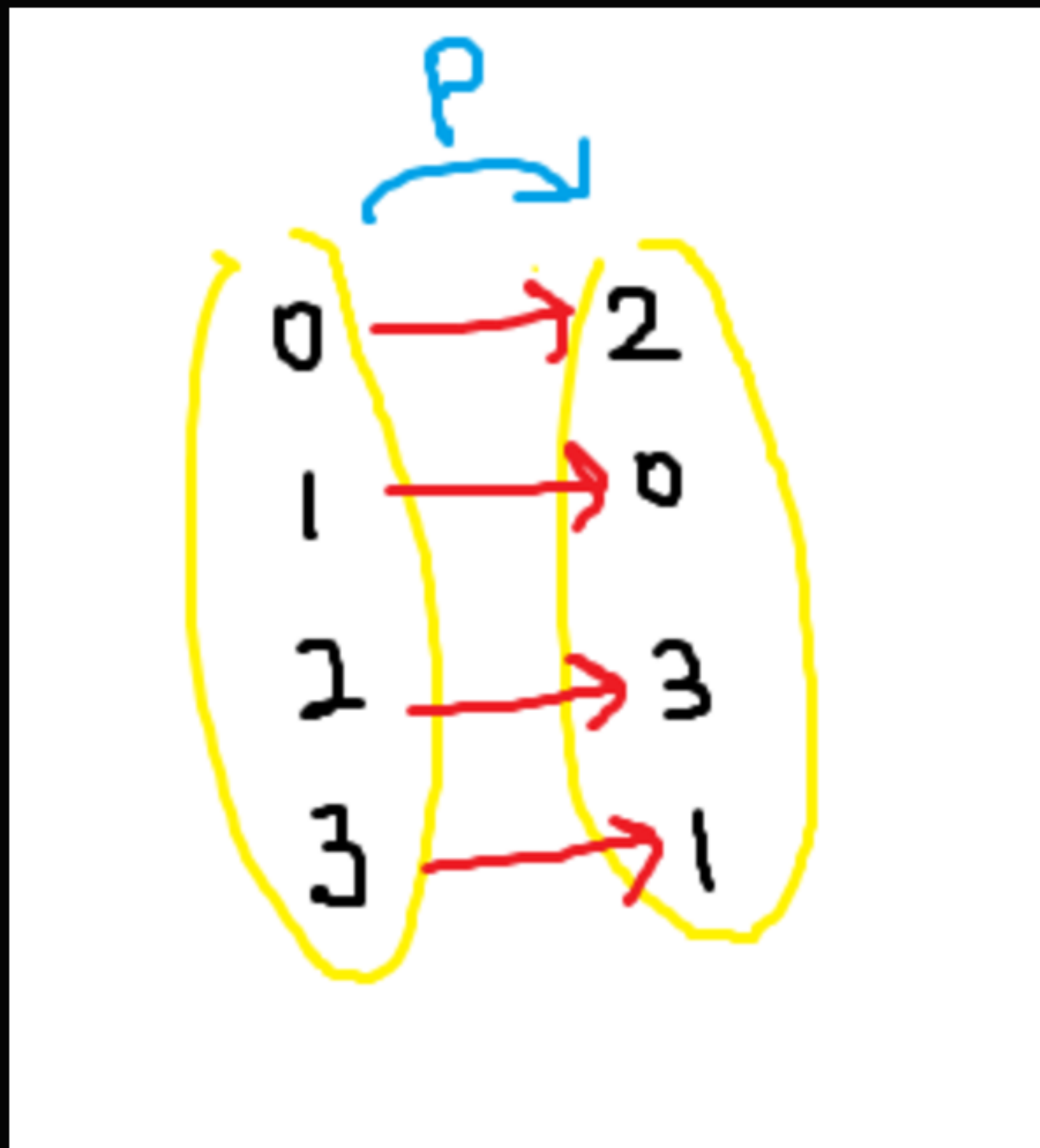


이제 중간 과정을 생략하면 P가 된다.

04

익명 함수

$$A = [2, 1, 3, 1], B = [1, 1, 2, 3]$$



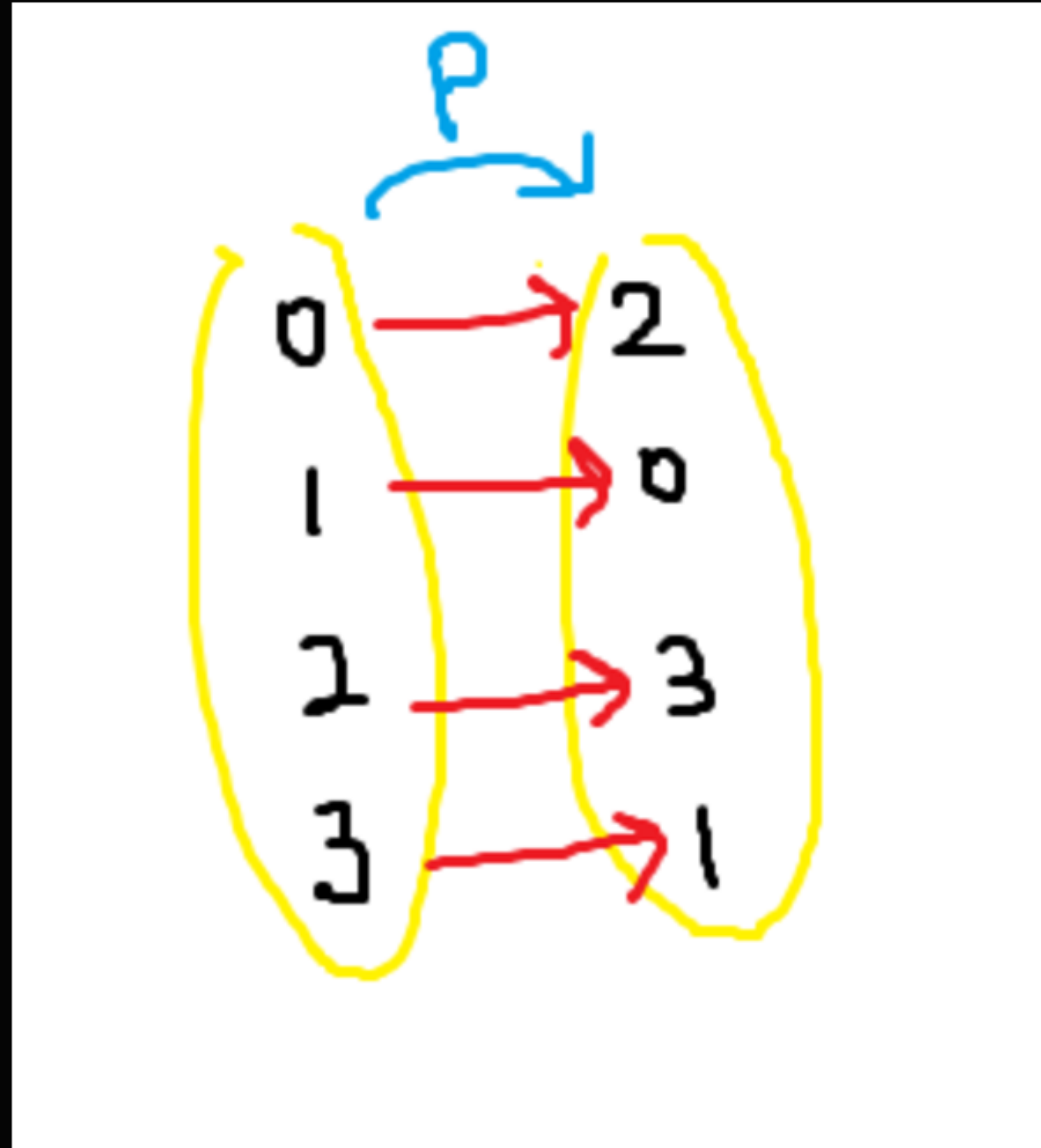
아하, 이제 좀 알겠다.

이 문제는 결국 원래 A의 각 원소에 정렬됐을 때의 인덱스를 매칭하라는 말과 같았다.

04

익명 함수

$$A = [2, 1, 3, 1], B = [1, 1, 2, 3]$$



[2, 1, 3, 1] 을 크기 순서대로 표시하면(단, 같다면 인덱스가 더 빠른)
[2, 0, 3, 1] 이다.

04

익명 함수

$A = [2, 1, 3, 1], B = [1, 1, 2, 3], P = [2, 0, 3, 1]$

그럼 이제 코드를 작성해보자.

우선 A 를 Z 로 변환하는 과정이 필요할 것이다.

$Z = [(A[i], i) \text{ for } i \text{ in range}(N)]$ # $Z == [(2, 0), (1, 1), (3, 2), (1, 3)]$

(값, 인덱스) 꼴의 튜플로 저장됨.

04

익명 함수

$A = [2, 1, 3, 1], B = [1, 1, 2, 3], P = [2, 0, 3, 1]$

정렬된 순서를 알아내기 위해 Z를 정렬한다.

`Z.sort()`

$Z == [(1, 1), (1, 3), (2, 0), (3, 2)]$

여전히 Z의 각 원소에는 원래 인덱스가 달려있다.

$A = [2, 1, 3, 1], B = [1, 1, 2, 3], P = [2, 0, 3, 1]$

원래 값 대신 정렬된 순서를 저장하고 원래 인덱스대로 복원한다..

$R = [(i, Z[i][1]) \text{ for } i \text{ in range}(N)]$ # (정렬된 인덱스, 원래 인덱스)

$R.sort(key=\lambda x: x[1])$ # 원래 인덱스를 기준으로 정렬

$R == [(0, 1), (1, 3), (2, 0), (3, 2)]$

정렬 후 $R == [(2, 0), (0, 1), (3, 2), (1, 3)]$

최종적으로 R은 $(P[i], i)$ 꼴의 튜플로 구성된다.

04

익명 함수

$A = [2, 1, 3, 1], B = [1, 1, 2, 3], P = [2, 0, 3, 1]$

이제 출력 양식에 맞게 출력하기만 하면 된다.

```
for i in range(N):  
    print(R[i][0], end=' ')
```

또는

```
for p, i in R:  
    print(p, end=' ')
```



```
import sys
 = sys.stdin.readline

N = int(input().strip())
A = list(map(int, input().split()))
Z = [(A[i], i) for i in range(N)]
Z.sort()

R = [(i, Z[i][1]) for i in range(N)]
R.sort(key=lambda x: x[1])

for i in range(N):
    print(R[i][0], end=' ')
```

다음 시간 예고: 시간복잡도(Big-O), 브루트포스, 백트래킹
(feat. 비트마스킹)
진짜 알고리즘의 시작이므로 많관부!



강의 설문조사

QR 제공 예정

Thank you For Listening

2주차 스터디를 들어주셔서 감사합니다.
뒷풀이에 참여하실 분들만 강의실에 남아주세요!

—

